

Podman & CLI Shipping Gotchas

- [Certain] **Port Mapping Syntax (-p host:container):** Reversing the port order is the most common trap. If Nginx listens on port 80 and you want it on host port 8080, you must write `-p 8080:80`.
- [Certain] **Host Networking Overrides (--network host):** When you pass `--network host`, any `-p` port mapping flags are completely ignored because the container shares the host's network namespace directly.
- [Certain] **Background Process Termination (-d):** Running lightweight images (alpine, busybox) in detached mode without a long-running foreground command causes them to immediately exit with code 0. Use a continuous command like `sleep infinity`.
- [Certain] **Lost Diagnostic Logs (--rm -d):** Combining `--rm` with `-d` wipes the container instantly upon failure. Always drop `--rm` during debugging so you can run `podman logs <container_id>`.
- [Certain] **SELinux Volume Denials (-v host:container:z):** Mounting host folders without the `:z` (private) or `:Z` (shared) relabeling flags on SELinux-enforced systems causes internal `Permission denied` errors.
- [Certain] **Rootless DNS Isolation:** Unlinked containers on the default bridge network cannot ping or reach each other by container name. You must explicitly create and assign a custom network (`podman network create <name>`).

Containerfile / Dockerfile Build Traps

- [Certain] **Distro Package Manager Mismatches:** Base images dictate the package manager. Using `yum` on `ubuntu` or `apt-get` on `alpine` breaks the build immediately:
 - Debian / Ubuntu `$\rightarrow$ apt-get update && apt-get install -y`
 - Alpine `$\rightarrow$ apk add --no-cache`
 - RHEL / Fedora `$\rightarrow$ dnf install -y`
- [Certain] **Cache-Busting Layer Order:** Copying your entire project (`COPY . /app`) *before* installing dependencies (`RUN npm install` or `RUN pip install`) forces a complete dependency re-install on every code change. Always copy manifest files (`package.json`, `requirements.txt`) and install dependencies *before* copying the remaining source code.
- [Certain] **Signal Swallowing (Shell vs. Exec Form):** Plain text commands (`CMD python app.py`) wrap the process inside `/bin/sh -c`, which blocks `SIGTERM` signals and prevents graceful shutdowns. Always use JSON array syntax (`CMD ["python", "app.py"]`).
- [Certain] **Destructive PATH Overrides:** Writing `ENV PATH=/app/bin` overwrites system utilities (`sh`, `ls`, `curl`). Always append to the existing path: `ENV PATH="/app/bin:$PATH"`.

- [Certain] **USER Permission Ordering:** Declaring a non-root user (`USER appuser`) before running root-required installation commands or copying files causes `Permission denied` build errors. Set ownership during copy (`COPY --chown=appuser:appuser . .`), run root installations first, and switch `USER` at the end.
- [Certain] **Single-Layer Image Size Bloat:** Running cleanup commands (`rm -rf /var/lib/apt/lists/*`) in a *separate* `RUN` step does not remove the underlying MB/GB from the final image. You must chain installations and cleanups within the exact same `RUN` command using `&&`.

Kubernetes Manifest Validation & Structure

- [Certain] **Deployment Selector Immutability:** `spec.selector.matchLabels` must strictly match `spec.template.metadata.labels`. Mismatches cause API validation rejection (`ValidationError`), and existing deployment selectors cannot be mutated without complete re-creation.
- [Certain] **API Version & Kind Pairs:** Mixing up group versions leads to schema failures. Memorize standard pairs:
 - Pod, Service, ConfigMap, Secret, PersistentVolumeClaim $\rightarrow$ `apiVersion: v1`
 - Deployment, DaemonSet, StatefulSet $\rightarrow$ `apiVersion: apps/v1`
- [Certain] **Service Endpoint Routing Failures:** If a Service returns no traffic, verify `kubectl get endpoints <service-name>`. Empty endpoints indicate that the Service's selector labels do not match the target Pod's `metadata.labels`, or `targetPort` doesn't match the container's listening port.
- [Certain] **Resource Starvation vs. Termination:** * If `resources.requests` exceed any node's capacity, the Pod hangs in `Pending (FailedScheduling)`.
 - If a running container breaches `resources.limits.memory`, the Linux kernel terminates it immediately (`OOMKilled` / Exit code 137).
- [Certain] **Missing Config References:** A Pod stays stuck in `CreateContainerConfigError` if a referenced `ConfigMapKeyRef` or `SecretKeyRef` does not exist or contains typos.
- [Certain] **Namespace Isolation:** Secrets, ConfigMaps, and PVCs are strictly namespace-scoped. A Pod cannot mount a Secret or PVC that exists in a different namespace.

Live Exam Diagnostic Commands

- [Certain] **Reading Previous Crash Logs:** When facing `CrashLoopBackOff`, standard logs often show empty restarts. Run `kubectl logs <pod-name> --previous` to read the exact stack trace from the dead container instance.

- [Certain] **Debugging Shell-less / Distroless Containers:** If `kubectl exec -it <pod> -- /bin/sh` fails, inject an ephemeral debug container directly into the running pod:

Bash

```
kubectl debug -it <pod-name> --image=busybox --target=<container-name>
```

- [Certain] **Testing Cluster DNS & Connectivity:** Launch an ephemeral throwaway pod to test internal routing:

Bash

```
kubectl run tmp --rm -it --image=busybox -- /bin/sh  
# Inside prompt: nslookup <service-name>
```

- [Certain] **Catching Typos Before Applying:** Run dry-run validation to catch indentation bugs, case-sensitivity typos (e.g., `imagePullpolicy` instead of `imagePullPolicy`), and misnested ports before modifying cluster state:

Bash

```
kubectl apply -f manifest.yaml --dry-run=server --validate=true
```